

Names: Rhilo Sotto, John Hayali, Eren Ugur

Report for Programming Assignment #3

Explanation of Implementation of Different Task Schedulers

EDF (*Earliest Deadline First*)

The EDF algorithm was implemented by incorporating the rule that the task with the earliest deadline has the highest priority. Specifically, each task in the schedule is created as an instance of a “task_instance” class and contains numerous attributes (e.g., name, parent, CPU_freq, period, arrival_time, deadline, remaining_time). If the user sets the second command-line parameter as “EDF,” then the functionality of the task_instance class is changed such that when two tasks are compared with each other using the less-than operator (<), the comparison operation returns True when the deadline of the task on the left-hand side is less than the deadline of the task on the right-hand side. This is necessary when the schedule needs to check whether the task with the earliest deadline in the ready queue has an earlier deadline than that of the running task (e.g., if ready_queue[0] < running_task). If this is the case, then the task is interrupted and placed in the ready queue, and the task that originally had the earliest deadline in the ready queue is set as the running task.

RM (*Rate Monotonic*)

The RM algorithm was carried out by applying the rule that the task with the lowest period has the highest priority. If the user sets the second command-line parameter as “RM,” then the functionality of the task_instance class is changed such that when two tasks are compared with each other using the less-than operator, the comparison operation returns True when the period of the task on the left-hand side is less than the period of the task on the right-hand side. This is necessary when the schedule needs to check whether the task with the lowest period in the ready queue has a smaller period than that of the running task (e.g., if ready_queue[0] < running_task). If this is the case, then the task is interrupted and placed in the ready queue, and the task that originally had the smallest period in the ready queue is set as the running task.

EE EDF (*Energy-Efficient Earliest Deadline First*)

The EE EDF scheduler extends the standard Earliest Deadline First algorithm by adding dynamic voltage and frequency scaling to reduce total energy use. Instead of running every job at the highest frequency, EE EDF tests all possible combinations of frequencies for each task. For every combination, it computes the total CPU utilization using each task’s WCET at that frequency and immediately rejects combinations where the utilization exceeds 1, since EDF cannot schedule them without missing deadlines. In the other combinations that are feasible, the scheduler will simulate a single complete hyperperiod of execution according to the EDF to

verify whether all the deadlines have been met and the amount of energy used. After evaluating all valid configurations, EE EDF selects the frequency assignment that results in the lowest energy consumption while still maintaining schedulability. EE EDF keeps the task ordering of EDF but tries to delay tasks to the maximum without deadline infringements, and uses as little energy as possible when the input set permits.

EE RM (Energy-Efficient Rate Monotonic)

EE RM applies the same DVFS optimization process as EE EDF but uses the Rate Monotonic scheduling policy during simulation. RM assigns priorities based only on task period, so each frequency combination must be tested under fixed-priority scheduling instead of deadline-based ordering. For every possible set of task frequencies, EE RM calculates per-task utilization, rejects over-utilized assignments, and then simulates one hyperperiod with RM to verify whether any deadlines are missed. Every feasible configuration is measured for total energy, and the one with the lowest consumption becomes the chosen frequency plan. The final full run uses RM with those selected frequencies. Functionally, EE RM slows down tasks just like EE EDF, but because RM has stricter schedule conditions, fewer low-frequency combinations tend to be feasible. In the provided test cases, both EE EDF and EE RM ultimately chose the highest frequency for all tasks because any slower configuration exceeded the schedulability bounds.

Results of the Input Cases

Outputs from using input1.txt

RM	EE RM	EDF	EE EDF
No feasible scheduling found.	No feasible scheduling found.	0 w4 1188 57 35.625 57 w2 1188 40 25.0 97 w5 1188 35 21.875 132 w3 1188 68 42.5 200 w4 1188 57 35.625 257 w2 1188 40 25.0 297 w3 1188 36 22.5 333 w1 1188 53 33.125 386 w5 1188 14 8.75 400 w4 1188 57 35.625 457 w5 1188 21 13.125 478 w2 1188 40 25.0 518 w3 1188 82 51.25 600 w4 1188 57 35.625 657 w5 1188 3 1.875	0 w4 1188 57 35.625 57 w2 918 50 22.35 107 w5 918 45 20.115 152 w3 1188 48 30.0 200 w4 1188 57 35.625 257 w2 918 50 22.35 307 w3 1188 56 35.0 363 w1 918 66 29.502 429 w4 1188 57 35.625 486 w5 918 45 20.115 531 w2 918 50 22.35 581 w3 1188 19 11.875 600 w4 1188 57 35.625 657 w5 918 3 1.341 660 w2 918 50 22.35

		660 w2 1188 40 25.0 700 w5 1188 32 20.0 732 w3 1188 22 13.75 754 w1 1188 46 28.75 800 w4 1188 57 35.625 857 w1 1188 7 4.375 864 IDLE IDLE 16 1.344 880 w2 1188 40 25.0 920 w5 1188 35 21.875 955 IDLE IDLE 45 3.78 592.00 6.10% 939	710 w5 918 42 18.774 752 w3 1188 85 53.125 837 w4 1188 57 35.625 894 w1 918 66 29.502 960 w2 918 50 22.35 539.22 0.00% 1000
--	--	---	--

The RM schedule using input1.txt is not feasible because task w1 failed to meet its deadline at time 520. The EE RM schedule is also not feasible because all of the 1024 cases for selecting the different frequency values for each task failed either due to CPU overutilization or a missed deadline.

For EDF, the total energy consumption during the execution was 592, the percentage of time spent idle was 6.10%, and the total system execution time was 939. For EE EDF, the total energy consumption during the execution was 539.22, the percentage of time spent idle was 0%, and the total system execution time was 1000. This indicates that the energy-efficient version of EDF saved 52.78 Joules compared to normal EDF and never experienced an idle period.

Outputs from using input2.txt

RM	EE RM	EDF	EE EDF
0 w5 1188 35 21.875	0 w5 1188 35 21.875	0 w5 1188 35 21.875	0 w5 918 45 20.115
35 w2 1188 40 25.0	35 w2 918 50 22.35	35 w2 1188 40 25.0	45 w2 648 67 20.569
75 w4 1188 57 35.625	85 w4 918 74 33.078	75 w4 1188 57 35.625	112 w4 648 103 31.621
132 w3 1188 104 65.0	159 w3 918 134 59.898	132 w3 1188 104 65.0	215 w3 918 134 59.898
236 w1 1188 53 33.125	293 w1 918 7 3.129	236 w1 1188 53 33.125	349 w1 918 66 29.502
289 IDLE IDLE 11 0.924	300 w5 1188 35 21.875	289 IDLE IDLE 11 0.924	415 w5 918 45 20.115
300 w5 1188 35 21.875	335 w2 918 50 22.35	300 w5 1188 35 21.875	460 w2 648 67 20.569
335 w2 1188 40 25.0	385 w1 918 59 26.373	335 w2 1188 40 25.0	527 w4 648 103 31.621
375 IDLE IDLE 75 6.3	444 IDLE IDLE 6 0.504	375 IDLE IDLE 75 6.3	630 w5 918 45 20.115
450 w4 1188 57 35.625	450 w4 918 74 33.078	450 w4 1188 57 35.625	675 w2 648 67 20.569
507 w3 1188 93 58.125	524 w3 918 76 33.972	507 w3 1188 93 58.125	742 w3 918 134 59.898
600 w5 1188 35 21.875	600 w5 1188 35 21.875	600 w5 1188 35 21.875	876 w1 918 66 29.502
635 w3 1188 5 3.125	635 w3 918 5 2.235	635 w3 1188 5 3.125	942 w5 918 45 20.115
640 w2 1188 40 25.0	640 w2 918 50 22.35	640 w2 1188 40 25.0	987 w2 648 67 20.569
680 w3 1188 6 3.75	690 w3 918 53 23.691	680 w3 1188 6 3.75	404.78 0.00% 1000
686 w1 1188 53 33.125	743 w1 918 66 29.502	686 w1 1188 53 33.125	

739 IDLE IDLE 161 13.524 900 w5 1188 35 21.875 935 w4 1188 25 15.625 960 w2 1188 40 25.0 491.37 24.70% 753	809 IDLE IDLE 91 7.644 900 w5 1188 35 21.875 935 w4 918 25 11.175 960 w2 918 50 22.35 441.18 9.70% 903	739 IDLE IDLE 161 13.524 900 w5 1188 35 21.875 935 w4 1188 25 15.625 960 w2 1188 40 25.0 491.37 24.70% 753	
--	--	--	--

For RM, the total energy consumption during the execution was 491.37, the percentage of time spent idle was 24.70%, and the total system execution time was 753. For EE RM, the total energy consumption during the execution was 441.18, the percentage of time spent idle was 9.70%, and the total system execution time was 903. This indicates that the energy-efficient version of RM saved 50.19 Joules compared to normal RM and experienced 15% less time spent idle.

For EDF, the total energy consumption during the execution was 491.37, the percentage of time spent idle was 24.70%, and the total system execution time was 753. For EE EDF, the total energy consumption during the execution was 404.78, the percentage of time spent idle was 0%, and the total system execution time was 1000. This indicates that the energy-efficient version of EDF saved 86.59 Joules compared to normal EDF and never experienced an idle period.

Optimal energy-efficient schedules

The program considers all combinations of the CPU frequency in all tasks to determine the most energy-efficient schedule. To each combination, the scheduler calculates the cumulative CPU utilization, taking into account the WCET of each task at the frequency assigned to the task. Combinations that exceed 100% utilization are immediately discarded since no feasible schedule can exist. Every remaining candidate is then fully simulated for one hyperperiod using the selected scheduling policy. The simulator uses a min-heap priority queue: under RM, tasks are ordered by period, and under EDF, they are ordered by deadline; the task at the top of the heap is always the one that should run next. If a new instance of a task is released before the previous instance has finished, the system flags a deadline miss and marks that frequency combination as infeasible. Among all feasible combinations, the program records the total energy consumed and selects the one with the lowest cost. This guarantees that the chosen EE schedule is globally optimal, not just locally or heuristically efficient. In the provided test cases, all slower combinations caused utilization to exceed the schedulable limit, so the optimal solution still required running tasks at 1188 MHz, but the search process correctly verified that no lower-energy configuration exists.